Enhancing LLM responses with RAG / Vector Search

Sebastien FLAESCH
Apr 2026

FOUR Js
The Power of Simplicity

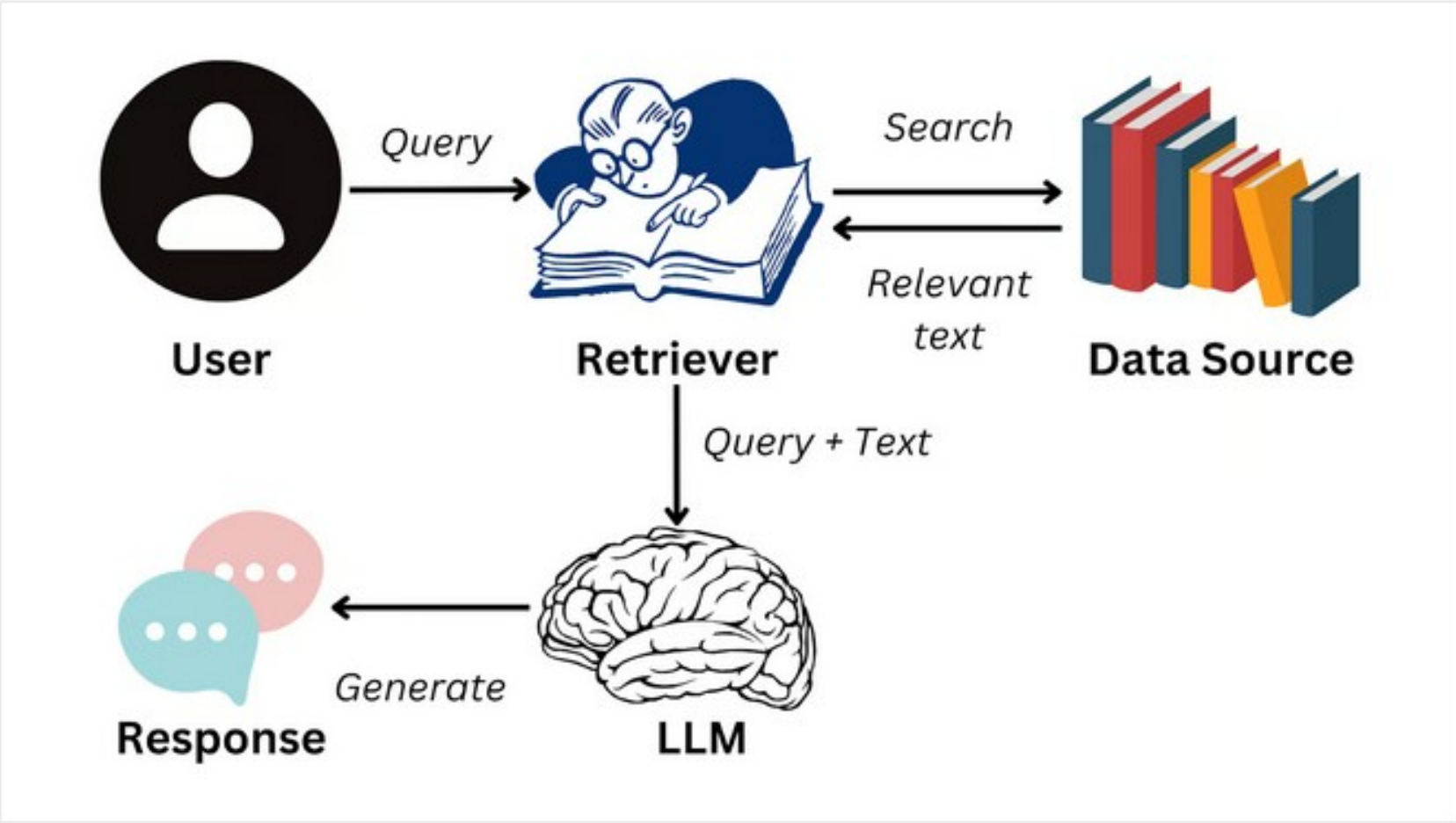
What is RAG ?

Retrieval **A**ugmented **G**eneration (RAG) empowers LLMs by enabling them to access external knowledge sources.

With RAG, LLMs can use **relevant information** to generate more accurate and contextually responses.

A crucial aspect of RAG is efficient information retrieval.

This is where **vector embeddings** and **specialized database technologies** come into play.



Source: https://www.datacamp.com/blog/what-is-retrieval-augmented-generation-rag

Benefits of RAG

- Minimize hallucinations: LLMs can generate inaccurate responses if they lack substantial data
  => RAG provides context to LLMs.
- Up-to-date information: LLMs are trained on public data up to a specific date
  => RAG provides fresh data.
- Confidentiality: Company-specific data must not be made public.
  => Keep enterprise data within your realm using local LLMs or in a cloud tenancy.

Efficient Retrieval Matters

Retrieval must be fast and semantically accurate

Vector Embeddings & **Vector Databases** enter in action!

Applies to structured and unstructured data:

Text, image, video, audio …

What are Vector Embeddings?

Numerical representation of the semantic meaning and relationships of real-world concepts or objects that are easily utilized by AI models and algorithms.

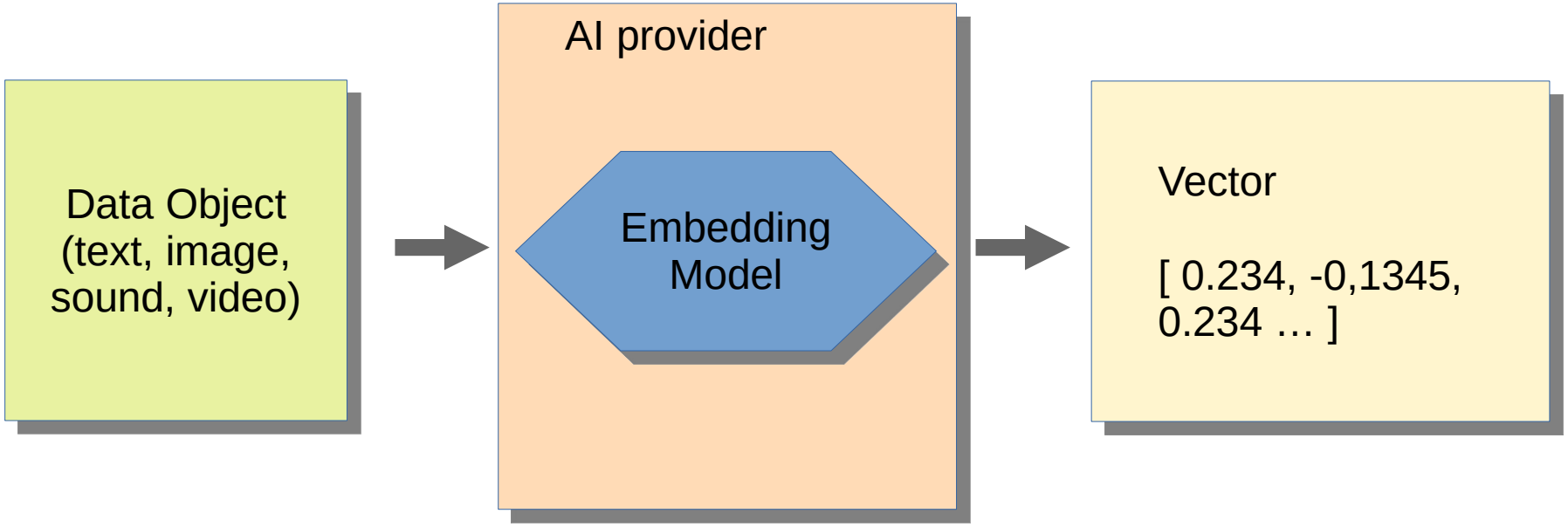
For example, a text:
"I have a dream …" => [0.234, 0.341, 1.580, …]

Dimension = Total number of features that are encoded in the vector representation.

A vector can be converted to/from a JSON array!

How to produce a vector?

Use **Embedding Models** from AI providers (can be public or private embedding model installed in-house):



Vector generation example

Using OpenAI API “text-embedding-ada-002” model:

```
curl https://api.openai.com/v1/embeddings \
  -H "Authorization: Bearer $OPENAI_API_KEY" \
  -H "Content-Type: application/json" \
  -d '{
    "input": "The food was delicious and the waiter...",
    "model": "text-embedding-ada-002",
    "encoding_format": "float"
  }'
```

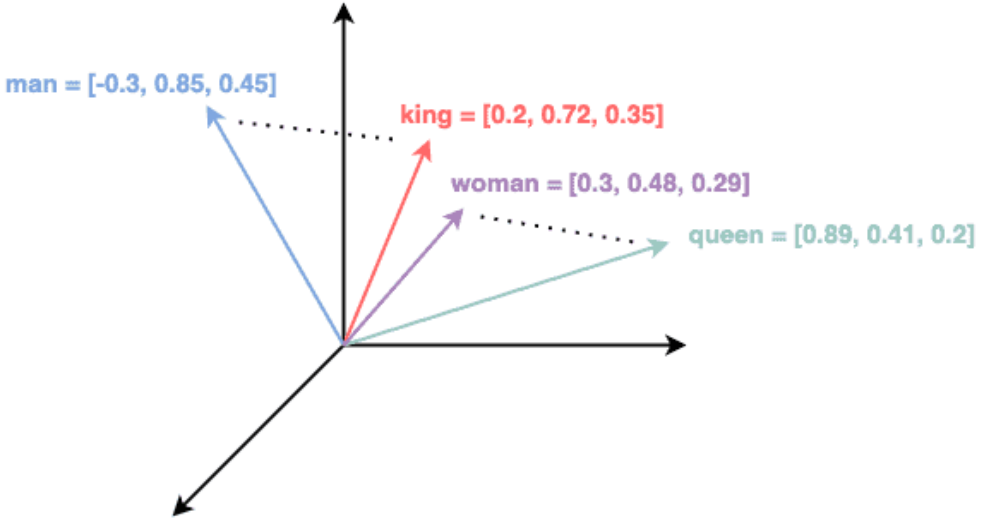
Vector Generation Result


```
{
  "object": "list",
  "data": [
    { "object": "embedding",
      "index": 0,
      "embedding": [0.002275,-0.009306,-0.00289,...]
    }
  ],
  "model": "text-embedding-ada-002-v2",
  "usage": { "prompt_tokens": 8, "total_tokens": 8 }
}
```

Vector Similarity

Similar objects are close to each other in a vector space

Ex: Semantically similar words in a 3-dimensional vector space:



Source: https://www.baeldung.com/cs/dimensionality-word-embeddings

Purpose of Vector Databases

- Store and fetch vector embeddings
- Support fast similarity search (with indexes)
- Purpose-built for high-performance retrieval

Pure Vector Databases (no SQL)

Chroma (https://www.trychroma.com/)

Pinecone (https://www.pinecone.io)

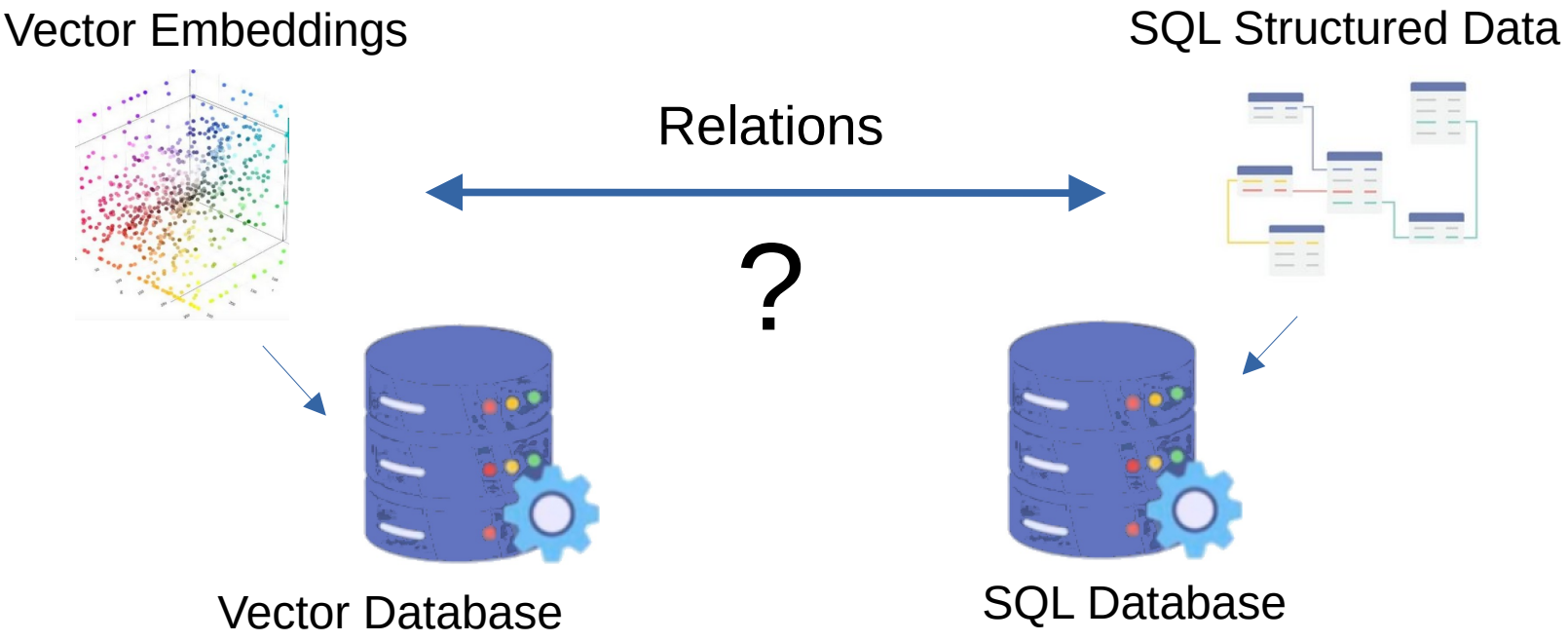
Weaviate (https://weaviate.io/)

Faiss (https://ai.meta.com/tools/faiss/)

… and more ...

Problem with Vector-only databases

=> Manages only vector data.



SQL Databases with Vector Search

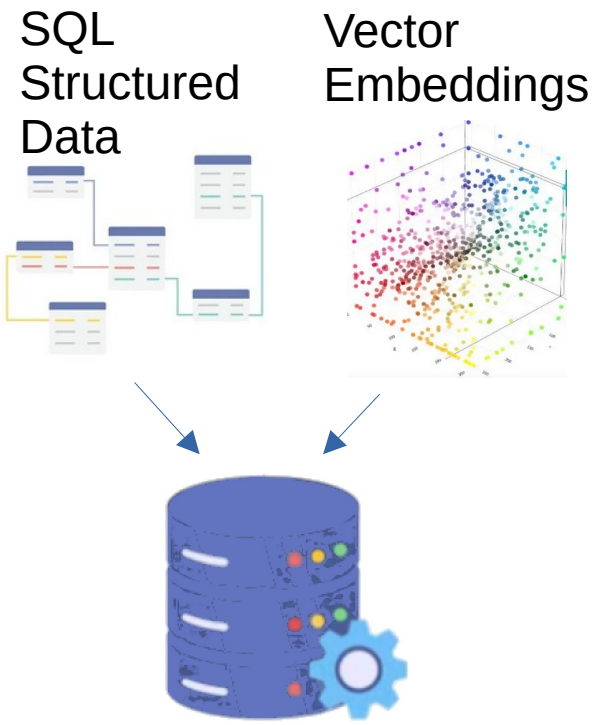
PostgreSQL
+ pgvector / pgvectorscale / pgai
extensions

Oracle 23ai
(**native** VECTOR type + functions)

Microsoft SQL Server 2025
(**native** VECTOR type + functions)

Benefits of SQL Integration

- Unified Data Management
  - Structured data + vector data together
  - No need for another software provider
- Scalability & Performance
  - Battle-tested SQL engines
- Data Integrity (ACID database engines)
  - Leverage transactional consistency



SQL table with vector column

```
CREATE TABLE customer (
    id          INTEGER NOT NULL PRIMARY KEY,
    name        VARCHAR(50) NOT NULL,
    prefs       VARCHAR(2000),
    prefs_emb   VECTOR(1024)
)
```

id	name	prefs	prefs_emb
4834	John STORM	Chocolate, Lemon, ...	[0.363,1.789,...]
7845	Phil Park	Coffee, Banana, ...	[0.984,-0.399,...]
7934	Judy STARN	Apple, Orange, ...	[0.254,1.435,...]
...	...	...	...

Powerful similarity search using SQL

Example with PostgreSQL + pgvector extension :

```
SELECT * FROM customer
 WHERE name LIKE 'Bar%'
 ORDER BY prefs_emb <=> ?::vector
```

This SQL orders rows by using the <=> operator of pgvector, computing the cosine distance between stored vectors and the vector passed as parameter, to get related / similar rows first.

Note the WHERE clause filtering rows on classical SQL data.

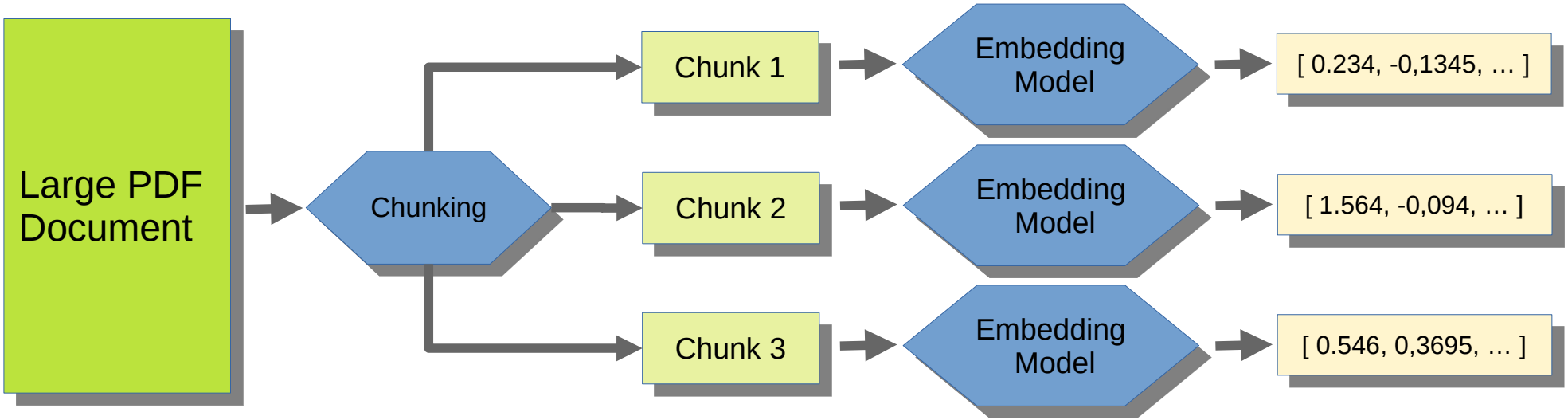
Chunking

- Vector embedding LLMs can only process a limited amount of text.
  Ex: OpenAI *text-embedding-ada-002* input token limit = 8191

- Chunking = Break down large pieces of text into smaller segments (= chunks)
  Typically: 1 paragraph or 1 page = 1 chunk of text

- Then compute a vector embedding for each chunk of text.

Chunking


Large PDF Document
Chunking
Chunk 1
Chunk 2
Chunk 3
Embedding Model
Embedding Model
Embedding Model
[0.234, -0,1345, …]
[1.564, -0,094, …]
[0.546, 0,3695, …]

SQL Vector Search + LLM

1) Must have **vector embeddings** created in your SQL database, bound to your structured or unstructured data.

SQL table with famous quotes made by celebrities
Martin Luther King : “I have a dream … “ => [0.234, 0.962, 1.362. …]
Nelson Mandela : “Education is … “ => [1.552, 0.842, 1.823, …]

2) Take user **contextual statement** and convert to a vector using same embedding engine that produced stored vectors.

“Famous quotes related to political concerns” => [0.111, 0.632, 0.985 ...]

3) Search similar data with SQL, using **vector similarity functions**

SELECT … FROM quotes ORDER BY emb <=> ?::context_vector
(here we use the ***<=>*** *operator to order rows by* ***cosine distance****)*

SQL Vector Search + LLM

4) Define **the role** of the LLM (system prompt)

You are a professor of history…

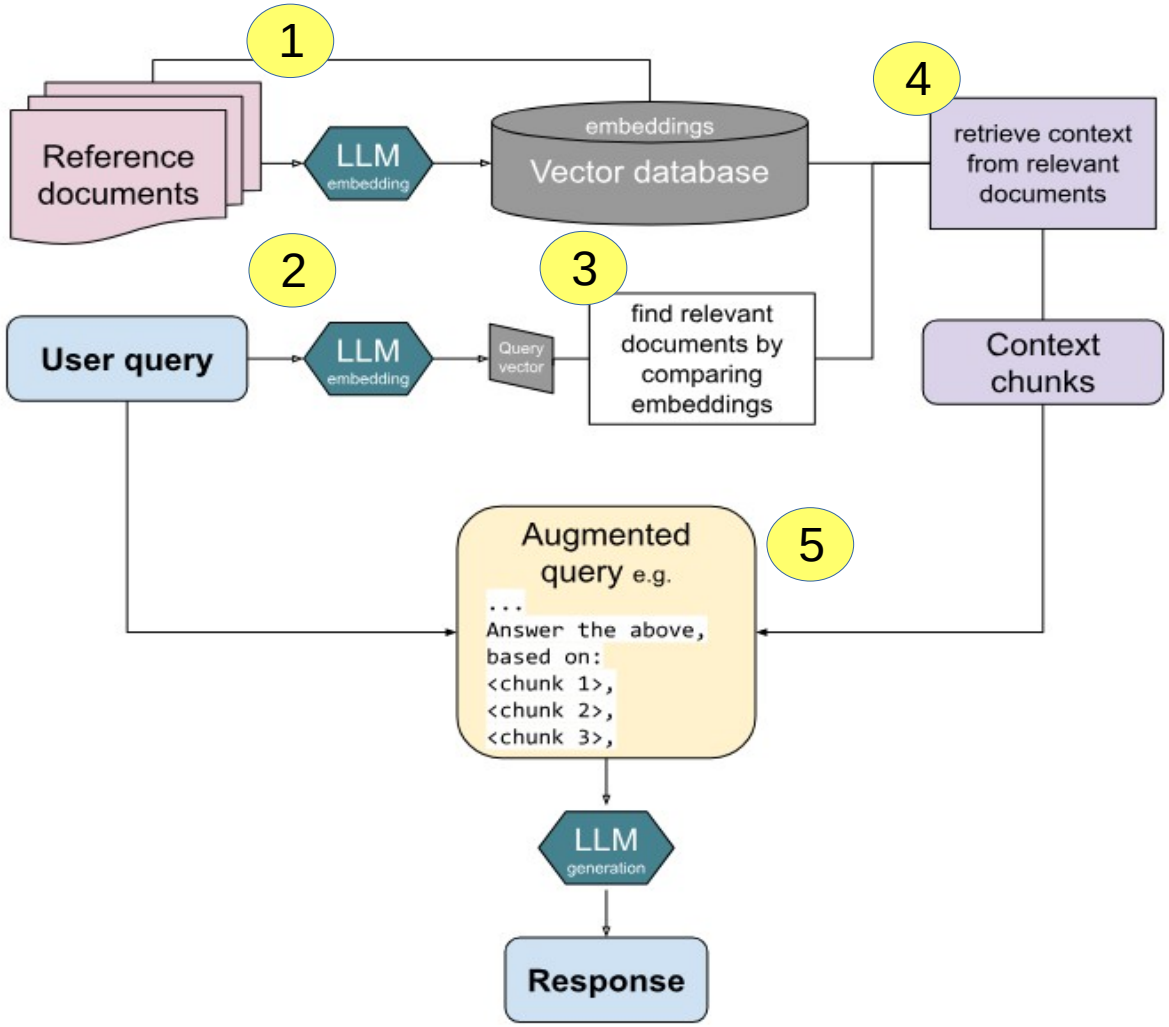
5) Provide LLM with the **context data** found by SQL query (assistant prompt)

Use the following relevant quotes:
Martin Luther King said: ...
Abraham Lincoln said: …
Nelson Mandela said: ...

6) Ask a **question** to LLM related to this context data:

Which politicians made a famous quote about racism?

7) LLM produces relevant **answer**, using the provided context data.



Source: https://blog.typingmind.com/rag-for-beginners/

Data Security Concerts with RAG

Never send sensitive data to public LLMs like ChatGPT!

Solutions:

- Setup your local on-prem LLM (if you have the computing power)
- Use private LLM engine provided by a Cloud Service Provider
  For ex: *Oracle Cloud Infrastructure Generative AI based on Cohere Large Language Model*
  https://docs.oracle.com/en/solutions/oci-generative-ai-llm/index.html

What FGL types for Vector Embeddings?

- STRING: To hold small / medium vectors (dim = 1024).
- TEXT: To hold very large vectors (dim > 1024)

To compute the size of the string representing a sequence of decimal numbers:

[0.026891274,0.022986012,-0.03174009,0.08214157,…]
< 12 chars >

For a vector with dimension 4096 => (4096 * 12) / 1024 = 48 Kb

Thank you for watching!

Q/A

FOUR Js
The Power of Simplicity